RxJS Observables and Operators

Complete Internal Mechanics Reference

Observable Core Mechanics — Observer — Subscribe — Subscription

Creation, Transformation, Flattening, Timing, Error Handling and Multicasting Operators

Compiled: 20 August 2026

Angular Learning — RxJS Deep Dive Session

Contents

1. Observable — Core Class Mechanics

2. The Producer Function

3. The Observer

4. The subscribe() Method

5. The Subscription Object

6. Full Internal Trace — Observable Lifecycle

7. Signals vs Observables

8. Real-Time Example — WebSocket Notification Stream (Angular)

9. Simpler Story Walkthrough — Delayed User Name Fetch

10. What "Stream of Data" Means

11. Why Unsubscribe Stops Emission

12. Why Not Unsubscribing Causes a Memory Leak

13. Generic Observable, Custom Producer, Custom Teardown

14. HttpClient and Observables

Operators — Creation

15. of()

16. from()

Operators — Flattening (Map Family)

17. switchMap

18. mergeMap

19. concatMap

20. exhaustMap

Operators — Combination

21. combineLatest

22. forkJoin

Operators — Transformation

23. map

24. filter

25. tap

26. scan

Operators — Timing Control

27. debounceTime

28. throttleTime

29. distinctUntilChanged

Operators — Error Handling

30. catchError

31. retry

Operators — Subscription Management

32. take

33. takeUntil

Operators — Multicasting

34. shareReplay

TOPIC 1 — OBSERVABLE — CORE CLASS MECHANICS

RxJS Core | Intermediate | Critical

---

1. Explanation

Analogy: Observable = a recipe, Subscribe = cooking it, Subscription = the stove you can turn off.

The Observable itself does nothing. It is inert — just a stored function. Nothing executes until subscribe() is called. That is the entire mental model.

Observable is a class. Its constructor takes one parameter — the producer function.

```
class Observable<T> {
  constructor(private producer: (observer: Observer<T>) => (() => void) | void) {}
}
```

producer gets stored as an instance property. Nothing runs yet. new Observable(fn) just creates an object holding fn — same as storing any callback on this.

2. The Producer Function

It is the function you write when you create an Observable. It defines what should happen when someone subscribes — fetch data, start an interval, listen for a DOM event, whatever.

```
const obs$ = new Observable<number>((observer) => {
  observer.next(1);
  observer.complete();
});
```

Here, the arrow function (observer) => {...} IS the producer. It takes one parameter — an Observer object — and uses it to emit values. It is just a plain function. Nothing special about it except the shape RxJS expects: it receives an observer, and it may return a cleanup function.

3. The Observer

Just three callbacks in a box. No magic.

```
interface Observer<T> {
  next: (value: T) => void;
  error: (err: any) => void;
  complete: () => void;
}
```

When you write observer.next(1) inside the producer function, you are literally calling the next callback the consumer gave you. The Observable is a middleman connecting the producer's calls to the consumer's callbacks.

The three callbacks are NOT part of the Observable class. They belong to whoever calls subscribe() — the consumer, not the Observable. You build an object with those three functions and pass it in as the argument to subscribe().

```
obs$.subscribe({
  next: (val) => console.log(val),
```

```
  error: (err) => console.log(err),
  complete: () => console.log('done'),
});
```

That object — { next, error, complete } — is the Observer. It is not a method of Observable, not stored on the class. It is just a plain object literal you construct at the call site and hand over as a parameter. subscribe(observer) receives it, then passes that same object straight into this.producer(observer).

So the flow of that object: you create it → pass to subscribe() → subscribe() forwards it unchanged to producer() → producer calls .next()/.complete() on it. The Observer is just data (three function references) being threaded through two function calls.

4. The subscribe() Method

subscribe() IS part of the Observable class. It's a method defined directly on the class — same as any class method.

```
class Observable<T> {
  constructor(private producer: (observer: Observer<T>) => (() => void) | void) {}

  subscribe(observer: Observer<T>): Subscription {
    const teardown = this.producer(observer);
    return new Subscription(teardown);
  }
}
```

You call it the normal way: obs$.subscribe(...). It has access to this.producer because it's a method on the same instance.

Every subscribe() call: (1) Runs producer(observer) synchronously, right there. (2) Captures whatever producer returns — that's your cleanup function. (3) Wraps it in a Subscription object and hands it back.

Two separate calls to subscribe() on the same Observable → producer runs twice, independently. No shared state unless you explicitly build one (that is what Subjects are for — different topic).

5. The Subscription Object

A completely separate class, unrelated to Observable and Observer.

```
class Subscription {
  private closed = false;
  constructor(private teardown?: () => void) {}

  unsubscribe(): void {
    if (this.closed) return;
    this.closed = true;
    this.teardown?.();
  }
}
```

It comes from subscribe()'s return statement. subscribe() calls this.producer(observer), captures whatever that call returns (the optional cleanup function), and wraps it in new

Subscription(teardown). That is the origin — subscribe() constructs it and hands it back to you as its return value.

```
const sub = obs$.subscribe({...});   // sub is a Subscription instance
```

unsubscribe() is called the same as any method on any class instance: sub.unsubscribe(). sub is a real Subscription object. unsubscribe is a method defined on that class. Calling it runs the method body — checks closed, flips it to true, calls this.teardown() if one exists.

6. Full Internal Trace — Observable Lifecycle

```
const obs$ = new Observable<number>((observer) => {
  observer.next(10);
  return () => console.log('cleanup');
});

const sub = obs$.subscribe({ next: v => console.log('got', v) });
sub.unsubscribe();
```

Step by step:

- new Observable(...) — stores producer function. Nothing runs. Nothing logs.
- obs$.subscribe({...}) called
- Inside subscribe, this.producer(observer) executes synchronously
- observer.next(10) runs → calls the next callback you passed → logs got 10
- producer returns the cleanup function → stored as teardown
- subscribe() returns a Subscription wrapping that teardown
- sub.unsubscribe() called → closed flips true → teardown() runs → logs cleanup

7. Signals vs Observables

	Signal	**Observable**
Execution	Eager — value exists immediately	Lazy — nothing runs until subscribe()
Consumption	Pull — you call signal() to read current value	Push — producer pushes values to you over time
Multiple reads	Always same value until changed	Each subscribe() = independent execution
Cleanup	None needed — GC handles it	Manual — unsubscribe() or it leaks
History	Only current value	Can emit sequence of values over time (stream)

Signals answer "what is the value now." Observables answer "here's a stream of values over time, and I'll tell you when I have one." That is why HTTP calls, WebSockets, and DOM events are Observables — they are inherently time-based, not a single current value.

ELI5 — Vending Machine Analogy

Analogy: A vending machine that only turns on when you press the button.

An Observable is a vending machine sitting there, unplugged. It does nothing by itself.

- **subscribe()** = you press the button. Only now does the machine turn on and start working.
- **The Observer** = you, holding out your hands, ready to catch whatever the machine gives you.
- **next()** = the machine drops a snack into your hands. It can drop multiple snacks, one after another.

- **complete()** = the machine says "no more snacks, I'm done."
- **The Subscription** = a cord plugged into the wall. If you want the machine to stop — even mid-snack — you pull the cord. That's unsubscribe().

If you never press the button, the machine never runs. If two people press the button, two separate machines start up — they do not share snacks. If you walk away without pulling the cord, the machine keeps running and dropping snacks into empty air forever. That is the leak.

TOPIC 8 — REAL-TIME EXAMPLE — WEBSOCKET NOTIFICATION STREAM

Angular Application | Intermediate | High

Real example: a live notification feed component using WebSockets. Same structure as before — Observable class, producer, subscribe, Observer, Subscription — but now wired to a real Angular use case: a service that streams live notifications, and a component that displays them and must stop listening when destroyed.

The Observable — constructor takes the producer function

```
// notification.service.ts
@Injectable({ providedIn: 'root' })
export class NotificationService {
  getLiveNotifications(): Observable<string> {
    return new Observable<string>((observer) => {
      const socket = new WebSocket('wss://api.example.com/notifications');

      socket.onmessage = (event) => observer.next(event.data);
      socket.onerror = (err) => observer.error(err);
      socket.onclose = () => observer.complete();

      return () => socket.close();   // cleanup function
    });
  }
}
```

new Observable<string>((observer) => {...}) — this constructor call stores the arrow function as producer. The WebSocket connection is NOT opened here. new WebSocket(...) only runs when producer actually executes — and producer only executes on subscribe(). This is why calling getLiveNotifications() alone does nothing — no network activity yet.

subscribe() called in the component

```
// notifications.component.ts
export class NotificationsComponent {
  private notificationService = inject(NotificationService);
  private destroyRef = inject(DestroyRef);

  ngOnInit() {
    this.notificationService.getLiveNotifications()
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe({
        next: (msg) => console.log('New notification:', msg),
        error: (err) => console.error('Socket error:', err),
        complete: () => console.log('Feed closed'),
      });
  }
}
```

.subscribe({...}) is called on the Observable instance returned by getLiveNotifications(). It is the same subscribe method defined on the Observable class — this.producer(observer) runs, which means this is the exact moment the WebSocket actually connects.

Full Trace — Realistic Angular Lifecycle

- Component created, ngOnInit() runs
- getLiveNotifications() called → returns an Observable instance, producer stored, no socket yet
- .pipe(takeUntilDestroyed(this.destroyRef)) wraps the Observable (adds auto-unsubscribe logic tied to component destruction — doesn't run anything yet either)
- .subscribe({...}) called → producer(observer) executes now
- Inside producer: new WebSocket(...) actually opens the connection
- socket.onmessage wired to call observer.next(data) whenever a message arrives
- Producer returns () => socket.close() → captured as teardown, wrapped in a Subscription
- Messages arrive over time → observer.next(data) fires each time → your next callback logs it
- Component gets destroyed (user navigates away) → DestroyRef fires → takeUntilDestroyed calls unsubscribe() on the underlying Subscription automatically
- unsubscribe() runs: closed flips true, teardown() executes → socket.close() → WebSocket connection actually closes

Why This Matters — Memory Leak Connection

Without takeUntilDestroyed, the component object gets garbage-collected-eligible, but socket.onmessage still holds a closure reference to observer → which closes over the component's next callback → the socket stays open, still receiving messages, still calling a callback tied to a component that no longer exists on screen. That is a live memory leak, not a theoretical one — an open WebSocket connection your server thinks is still active, doing work for nobody. unsubscribe() calling socket.close() is the only thing that tells the server "stop sending, nobody's listening anymore."

TOPIC 9 — SIMPLER STORY WALKTHROUGH — DELAYED USER NAME FETCH

Angular Application | Beginner-Intermediate | High

The story: a service that gives you a user's name after a short delay.

Part 1 — The Service, and What new Observable() Actually Returns

```
@Injectable({ providedIn: 'root' })
export class UserService {
  getUserName(): Observable<string> {
    return new Observable<string>((observer) => {
      console.log('Producer started - fetching...');

      const timerId = setTimeout(() => {
        observer.next('Naveen');
        observer.complete();
      }, 2000);

      return () => {
        console.log('Cleanup - cancelling fetch');
        clearTimeout(timerId);
      };
    });
  }
}
```

getUserName() is called. Inside it, new Observable<string>((observer) => {...}) executes. This does exactly one thing: it creates an Observable instance and stores the arrow function (observer) => {...} inside it, on a property called producer. That's all. The arrow function body — the console.log, the setTimeout — does NOT run. It's just sitting there, saved.

That instance is what gets returned out of getUserName(). So when a component calls this.userService.getUserName(), what it gets back is a plain object: an Observable holding an unexecuted function. Nothing has fetched anything. No timer is running. It's inert — a recipe, not a meal.

Part 2 — The Component Subscribes

```
@Component({...})
export class ProfileComponent implements OnInit {
  private userService = inject(UserService);
  private sub!: Subscription;

  ngOnInit() {
    const userName$ = this.userService.getUserName();
    // still nothing has happened - userName$ is just the Observable

    this.sub = userName$.subscribe({
      next: (name) => console.log('Got name:', name),
      complete: () => console.log('Stream complete'),
    });
  }

  ngOnDestroy() {
```

```
    this.sub.unsubscribe();
  }
}
```

Now follow the story from .subscribe(...) onward, in order:

Step 1. userName$.subscribe({ next, complete }) is called. The object { next, complete } is built right there in the component — this is the Observer. It's just two functions in a box, nothing more.

Step 2. Internally, subscribe() takes that Observer object and immediately calls this.producer(observer) — passing your Observer straight into the function that was sitting dormant since Part 1.

Step 3. That producer function now actually runs, for the first time. console.log('Producer started - fetching...') fires. setTimeout starts a real 2-second timer.

Step 4. The producer function reaches its return statement — () => { clearTimeout(timerId); ... } — and returns that cleanup function back out.

Step 5. Back inside subscribe(), that returned cleanup function is grabbed and wrapped into new Subscription(cleanupFn). This new Subscription object is what subscribe() hands back to the component — which is why this.sub = userName$.subscribe(...) captures it.

Step 6. Two seconds pass. The setTimeout callback fires. It calls observer.next('Naveen'). Because observer is the exact same object the component built in Step 1, this is really just calling the component's next function directly — console.log('Got name:', name) runs, printing Got name: Naveen.

Step 7. Immediately after, observer.complete() runs — calls the component's complete function — prints Stream complete.

Step 8. Now imagine instead the user navigates away before the 2 seconds are up. Angular calls ngOnDestroy(), which calls this.sub.unsubscribe().

Step 9. Inside unsubscribe(): it checks closed (false), flips it to true, then calls the stored cleanup function from Step 4 — clearTimeout(timerId) runs. console.log('Cleanup - cancelling fetch') prints. The pending timer is killed before it ever calls next.

The shape of the whole story: the service builds and hands over an inert Observable. The component's .subscribe() call is the trigger that wakes the producer up, and in doing so, exchanges two things with it — the component's {next, complete} gets fed into the producer so the producer can call back into the component later, and the producer's cleanup function gets carried out wrapped in a Subscription so the component can shut things down early. Everything that happens after — the delayed next, the complete, or the early unsubscribe — is just those two exchanged functions being invoked at different points in time.

TOPIC 10 — WHAT "STREAM OF DATA" MEANS

RxJS Core Concept | Intermediate | Critical

A single HTTP call gives you one value, once, and it's done — like a vending machine that drops exactly one snack and shuts off. A stream is different: the producer can call observer.next() multiple times, spread across time, before it ever calls complete(). That's the whole definition. Not one value — a sequence of values, arriving whenever the producer decides to emit one.

```
new Observable<number>((observer) => {
  observer.next(1);   // emission 1 - happens now
  setTimeout(() => observer.next(2), 1000);   // emission 2 - 1 sec later
  setTimeout(() => observer.next(3), 2000);   // emission 3 - 2 sec later
  // never calls complete() - could keep going forever
});
```

"Stream" just means: the producer is allowed to call next() as many times as it wants, whenever it wants. A mouse-move Observable, a WebSocket, a polling interval — none of these have "one answer." They have an ongoing sequence. That's why the word "stream" fits — data flowing continuously, not a single delivery.

11. Why Unsubscribe Stops Emission

Go back to the WebSocket example. The producer did this:

```
socket.onmessage = (event) => observer.next(event.data);
```

Every time a message arrives on the socket, this line runs. It calls observer.next(...) — which is really calling the component's next callback, because observer IS the object the component built.

Now, unsubscribe() does NOT reach into the socket and turn it off by itself. It only does one thing: runs the teardown function the producer returned.

```
return () => socket.close();
```

So the real chain is: unsubscribe() → calls teardown() → calls socket.close() → the actual WebSocket connection dies → socket.onmessage can never fire again → observer.next() can never be called again.

The Observable "stops emitting" only because you, the producer author, wrote a teardown function that kills the real underlying source. RxJS doesn't magically stop it. If your teardown function did nothing (return () => {}), calling unsubscribe() would do nothing — the socket would keep streaming forever.

This is the same for setInterval:

```
const id = setInterval(() => observer.next(Date.now()), 1000);
return () => clearInterval(id);   // this is what actually stops emissions
```

clearInterval(id) is the real stop switch. unsubscribe() is just the button that triggers it.

12. Why Not Unsubscribing Causes a Memory Leak

If you never call unsubscribe(), that teardown function never runs. So clearInterval never fires, or socket.close() never fires. The real underlying thing — the timer, the socket, the event listener — keeps running exactly as if nothing happened. It has no idea your Angular component was destroyed. It's not tied to the component's lifecycle at all; it's tied to the browser/OS, completely separate from Angular.

Here's the chain that makes it a leak specifically, not just wasted work:

```
this.sub = userName$.subscribe({
  next: (name) => this.username = name,   // 'this' - closes over the component
});
```

That next callback closes over this — the component instance. The running interval/socket holds a reference to observer, which holds a reference to this next function, which holds a reference to this (the component).

So even after Angular removes the component from the DOM and would normally garbage-collect it, it can't — because the still-running interval/socket is holding a live reference to it through that chain. The component becomes a zombie: invisible on screen, but still sitting in memory, still receiving next() calls, still doing work, forever, because nothing ever told the interval to stop.

One sentence version: unsubscribe() only stops emissions because it runs your teardown function, and your teardown function is the thing that kills the real source (timer/socket/listener) — skip unsubscribe(), and that real source keeps running and keeps a live reference to your component, which is the leak.

TOPIC 13 — GENERIC OBSERVABLE, CUSTOM PRODUCER, CUSTOM TEARDOWN

RxJS Core Concept | Intermediate | Critical

Observable — one generic class, reused everywhere. Same subscribe(), same Subscription mechanics, no matter what's underneath.

Producer — different every time, written specifically for whatever real thing it wraps. HTTP call, WebSocket, setInterval, DOM event listener — each producer knows how to start that specific thing and, critically, how to stop it.

Teardown — the producer's own "kill switch" for that specific thing.

- HTTP → teardown calls .abort() on the request
- WebSocket → teardown calls socket.close()
- setInterval → teardown calls clearInterval(id)
- DOM event → teardown calls removeEventListener(...)

unsubscribe() itself does nothing type-specific. It's generic — it just calls whatever function the producer handed it. It has no idea if it's closing a socket or clearing a timer. All the real, type-specific "stop this" logic lives inside the teardown function, written by whoever authored that particular Observable.

Precise correction on phrasing: there is really only one Observable class. What differs is only the producer function passed into it. The class itself never changes; only the behavior you hand it does.

14. HttpClient and Observables

When HttpClient methods return an Observable, it means Angular constructs the exact same generic Observable class — with Angular's own producer function plugged in. Simplified version of what HttpClient.get() does internally:

```
// simplified version of what Angular does internally
get(url: string): Observable<any> {
  return new Observable((observer) => {
    const controller = new AbortController();

    fetch(url, { signal: controller.signal })
      .then(response => response.json())
      .then(data => {
        observer.next(data);
        observer.complete();
      })
      .catch(err => observer.error(err));

    return () => controller.abort();   // teardown - cancels the in-flight request
  });
}
```

Same Observable class. Same subscribe() mechanics. Same Subscription object coming back. The only thing Angular supplies that's different from a WebSocket or timeout example is the

producer function's body — this one calls fetch() instead of opening a socket, and its teardown calls .abort() instead of .close() or clearTimeout().

```
this.http.get('/api/users').subscribe({
  next: (users) => console.log(users),
});
```

.subscribe() triggers Angular's producer to run → fetch() actually fires now (not before) → when the response resolves, observer.next(data) calls your next callback → observer.complete() follows immediately after, since an HTTP call is a single emission, not an ongoing stream → you get back a Subscription wrapping the abort() teardown, which is why calling unsubscribe() before the response arrives actually cancels the real network request.

TOPIC 15 — of() — CREATION OPERATOR

RxJS Operator - Creation | Beginner | Medium

---

of(...) — emits each argument exactly as given, then completes.

```
function of<T>(...args: T[]): Observable<T> {
  return new Observable<T>((observer) => {
    for (const item of args) {
      observer.next(item);
    }
    observer.complete();
  });
}
```

of(1, 2, 3) → producer loops over [1, 2, 3] → calls observer.next(1), next(2), next(3), then complete(). Three emissions, synchronous, all in one tick, then done.

of([1, 2, 3]) (array passed as ONE argument) → producer treats the whole array as a single item → observer.next([1, 2, 3]) → one emission, the array itself.

Rule: of never unpacks anything. It emits exactly what you hand it, argument by argument.

TOPIC 16 — from() — CREATION OPERATOR

RxJS Operator - Creation | Beginner | Medium

---

from(...) — unpacks an iterable/promise into individual emissions.

```
function from<T>(source: T[] | Promise<T>): Observable<T> {
  return new Observable<T>((observer) => {
    if (Array.isArray(source)) {
      for (const item of source) {
        observer.next(item);
      }
      observer.complete();
    } else {
      // it's a Promise
      source.then(
        (value) => { observer.next(value); observer.complete(); },
        (err) => observer.error(err)
      );
    }
  });
}
```

from([1, 2, 3]) → producer sees an array → loops and calls next(1), next(2), next(3), complete(). Same end result as of(1,2,3) here, but the mechanism is different: from looked inside the array and unpacked it. If you'd done from([[1,2,3]]) — array of one array — it unpacks one level, emitting the inner array as a single next().

from(fetch('/api/data')) → producer sees a Promise → calls .then() on it → when the promise resolves, observer.next(value) then complete(). This is literally how you turn a Promise into an Observable — the producer bridges Promise's .then() callback into observer.next().

The Actual Distinction, Mechanically

of never inspects what you pass — it treats every argument as one opaque item to emit. from inspects the single argument you pass — if it's iterable (array, string, Map) or a Promise, it knows how to walk through it and emit its contents piece by piece.

```
of('hello')        // one emission: 'hello'
from('hello')      // five emissions: 'h','e','l','l','o' - string is iterable, unpac
ked char by char
```

That's the confusion point resolved at the mechanism level: from's producer contains unpacking logic (a loop/iterator walk or a .then()), of's producer just iterates its arguments list and re-emits each one untouched.

TOPIC 17 — switchMap, mergeMap, concatMap — SHARED SKELETON

RxJS Operator - Flattening | Advanced | Critical

Same job, different internal producer strategy for handling overlapping inner Observables. All three do one thing structurally: take a source Observable, and for each value it emits, call a function that returns a new inner Observable, then subscribe to that inner one. The only difference between the three is how they handle a new outer emission arriving while a previous inner subscription is still active.

```
function customMap(source: Observable<any>, project: (val: any) => Observable<any>) {
  return new Observable((observer) => {
    let outerSub: Subscription;
    let innerSub: Subscription | null = null;

    outerSub = source.subscribe({
      next: (outerValue) => {
        const innerObservable = project(outerValue);   // build the inner Observable
        // *** here's where the three operators diverge ***
        innerSub = innerObservable.subscribe({
          next: (innerValue) => observer.next(innerValue),
        });
      },
    });

    return () => {
      outerSub.unsubscribe();
      innerSub?.unsubscribe();
    };
  });
}
```

The marked line is the only thing that changes across the three operators.

Shared Setup for Live Traces — Search-As-You-Type

```
// Outer source: simulates keystrokes
// 'a' at t=0ms, 'ab' at t=100ms, 'abc' at t=350ms

// Inner Observable: simulates an HTTP search call taking 200ms
function search(term: string): Observable<string> {
  return new Observable((observer) => {
    console.log(`[${term}] HTTP request started`);
    const timerId = setTimeout(() => {
      observer.next(`results for "${term}"`);
      observer.complete();
    }, 200);

    return () => {
      clearTimeout(timerId);
      console.log(`[${term}] HTTP request ABORTED`);
    };
  });
}
```

Same outer timeline used in all traces: 'a' at t=0, 'ab' at t=100, 'abc' at t=350. Each inner call takes 200ms to resolve.

TOPIC 17.1 — switchMap

RxJS Operator - Flattening | Advanced | Critical

Kills the previous inner subscription before starting a new one.

```
next: (outerValue) => {
  innerSub?.unsubscribe();   // cancel whatever inner was running
  const innerObservable = project(outerValue);
  innerSub = innerObservable.subscribe({ next: v => observer.next(v) });
},
```

New outer value arrives → old inner subscription's teardown runs immediately (real cancellation — if it's an HTTP call, abort() fires) → then a fresh inner subscription starts. Only one inner subscription alive at a time, always the latest.

Angular use case: search-as-you-type. Every keystroke fires an HTTP call; you only want the response to the latest keystroke. Old requests get aborted mid-flight.

```
keystrokes$.pipe(switchMap(term => search(term))).subscribe(v => console.log('OUTPUT:
', v));
```

Time	Event
t=0	Outer emits 'a'. No inner running yet. switchMap calls search('a'), subscribes. [a] HTTP request started
t=100	Outer emits 'ab'. Inner for 'a' still active (started t=0, due t=200). switchMap immediately calls unsubscribe() on it → teardown runs → [a] HTTP request ABORTED. Then subscribes to search('ab'). [ab] HTTP request started
t=300	Inner for 'ab' (started t=100) completes on schedule. observer.next('results for "ab"') fires → OUTPUT: results for "ab"
t=350	Outer emits 'abc'. Inner for 'ab' already completed at t=300 - nothing to cancel. switchMap subscribes to search('abc'). [abc] HTTP request started
t=550	Inner for 'abc' completes → OUTPUT: results for "abc"

Result for 'a' never appears. It was killed mid-flight. Total output: 'ab', 'abc'.

TOPIC 17.2 — mergeMap

RxJS Operator - Flattening | Advanced | Critical

Never cancels; lets all inner subscriptions run in parallel.

```
next: (outerValue) => {
  const innerObservable = project(outerValue);
  const sub = innerObservable.subscribe({ next: v => observer.next(v) });
  activeInnerSubs.push(sub);   // just tracked, never killed early
},
```

New outer value arrives → previous inner subscription is left completely alone, still running → a new inner subscription starts alongside it. Multiple inner subscriptions alive simultaneously, all emitting into the same output stream whenever they each resolve, in whatever order they finish.

Angular use case: uploading multiple files, want all upload requests to fire concurrently and each report progress independently, regardless of order.

```
keystrokes$.pipe(mergeMap(term => search(term))).subscribe(v => console.log('OUTPUT:'
, v));
```

Time	Event
t=0	Outer emits 'a'. Subscribe to search('a') - runs independently. [a] HTTP request started
t=100	Outer emits 'ab'. Inner for 'a' is untouched, still running. New inner subscribed in addition. [ab] HTTP request started - now 2 requests in flight
t=200	Inner for 'a' completes → OUTPUT: results for "a"
t=300	Inner for 'ab' completes → OUTPUT: results for "ab"
t=350	Outer emits 'abc'. Both previous inners already done. Subscribe to search('abc'). [abc] HTTP request started
t=550	Inner for 'abc' completes → OUTPUT: results for "abc"

All three results appear — nothing cancelled, nothing queued. In this timeline they happen to resolve in order, but if search('a') were slower than search('ab'), output order could flip — mergeMap gives no ordering guarantee, only concurrency.

TOPIC 17.3 — concatMap

RxJS Operator - Flattening | Advanced | Critical

Queues; waits for the current inner to complete before starting the next.

```
next: (outerValue) => {
  queue.push(outerValue);
  if (!currentlyRunning) runNext();
},

function runNext() {
  const outerValue = queue.shift();
  const innerObservable = project(outerValue);
  currentlyRunning = true;
  innerObservable.subscribe({
    next: v => observer.next(v),
    complete: () => { currentlyRunning = false; runNext(); },  // only starts next af
ter current finishes
  });
}
```

New outer value arrives while one inner is still active → it's queued, not started. Only when the current inner Observable calls complete() does the next queued one begin. Only one inner subscription alive at a time, but nothing gets cancelled — everything runs, strictly in order.

Angular use case: sequential save operations that must happen in order — e.g. saving form section A, then B, then C, where B must not start until A's save request has actually finished.

```
keystrokes$.pipe(concatMap(term => search(term))).subscribe(v => console.log('OUTPUT:
', v));
```

Time	Event
t=0	Outer emits 'a'. Nothing running → starts immediately. [a] HTTP request started
t=100	Outer emits 'ab'. 'a' is still running → 'ab' gets queued, NOT started
t=200	Inner for 'a' completes → OUTPUT: results for "a". Queue checked → 'ab' dequeued, started now. [ab] HTTP request started
t=350	Outer emits 'abc'. 'ab' still running (started t=200, due t=400) → 'abc' gets queued
t=400	Inner for 'ab' completes → OUTPUT: results for "ab". Queue checked → 'abc' dequeued, started. [abc] HTTP request started
t=600	Inner for 'abc' completes → OUTPUT: results for "abc"

All three results appear, strictly in order, nothing cancelled — but the last result lands 50ms later than switchMap/mergeMap would've given it, because 'abc' had to wait its turn behind 'ab'.

Summary — switchMap vs mergeMap vs concatMap

Operator	On new outer value, if inner is still active
switchMap	Cancel it, start new
mergeMap	Ignore it, start new anyway (both run)
concatMap	Wait - queue new, don't start until current completes

TOPIC 20 — exhaustMap

RxJS Operator - Flattening | Advanced | Critical

The fourth flattening operator, sibling to switchMap/mergeMap/concatMap. Same skeleton as those — outer subscribes, projects each value to an inner Observable — but the policy for "what happens if inner is already running" is: ignore the new outer value completely. No cancel, no queue — just drop it.

```
function customExhaustMap<T, R>(source: Observable<T>, project: (val: T) => Observabl
e<R>): Observable<R> {
  return new Observable<R>((observer) => {
    let innerActive = false;
    let outerSub: Subscription;
    let innerSub: Subscription | null = null;

    outerSub = source.subscribe({
      next: (outerValue) => {
        if (innerActive) {
          return;   // an inner is already running - drop this outer value entirely
        }
        innerActive = true;
        const innerObservable = project(outerValue);
        innerSub = innerObservable.subscribe({
          next: (innerValue) => observer.next(innerValue),
          complete: () => {
            innerActive = false;   // only NOW does a new outer value get accepted
            innerSub = null;
          },
        });
      },
      complete: () => observer.complete(),
    });

    return () => {
      outerSub.unsubscribe();
      innerSub?.unsubscribe();
    };
  });
}
```

The innerActive boolean is the entire mechanism — same gate pattern as throttleTime's inCooldown, just triggered by inner-completion instead of a timer.

Live Trace — Login Button

```
function login(): Observable<string> {
  return new Observable((observer) => {
    console.log('LOGIN request sent');
    const id = setTimeout(() => {
      observer.next('login success');
      observer.complete();
    }, 200);
    return () => clearTimeout(id);
  });
}
```

Outer source: click at t=0, click at t=100, click at t=250. Inner login() call takes 200ms.

Time	Event
t=0	Click fires. innerActive is false → proceed. innerActive = true. login() subscribed → LOGIN request sent.
t=100	Click fires. innerActive is true (still running, due t=200) → dropped immediately, project() never even called, no second request sent.
t=200	Inner completes → observer.next('login success') fires → prints login success. complete handler runs → innerActive = false, innerSub = null.
t=250	Click fires. innerActive is now false (window reopened at t=200) → proceed. New login() subscribed → LOGIN request sent again.

Result: exactly 2 login requests sent, not 3. The click at t=100 vanished with zero trace — no cancellation log, no queued execution, because project(outerValue) was never even called for it.

Contrast, Mechanically, on This Same Timeline

- switchMap would've cancelled request #1 at t=100 and started a fresh one - wrong for login, you'd get a race of two auth attempts.
- concatMap would've queued the t=100 click and fired a second login right after the first completes at t=200 - still sends 2 requests, but for the wrong reason (double-submits the same click).
- exhaustMap is correct here because it structurally prevents a second submission while the first is still in flight, without needing extra state like a disabled flag on the button.

TOPIC 21 — combineLatest

RxJS Operator - Combination | Advanced | High

Combinator over multiple independent outer Observables (not nested, unlike the Map operators). Different shape: instead of one outer stream feeding inner Observables, you give these multiple Observables up front, and the producer subscribes to all of them simultaneously.

Emits whenever ANY source emits, using the latest value from each.

```
function customCombineLatest(sources: Observable<any>[]): Observable<any[]> {
  return new Observable((observer) => {
    const latestValues: any[] = new Array(sources.length);
    const hasEmitted: boolean[] = new Array(sources.length).fill(false);
    const subs: Subscription[] = [];

    sources.forEach((source, index) => {
      const sub = source.subscribe({
        next: (value) => {
          latestValues[index] = value;
          hasEmitted[index] = true;

          // only emit once ALL sources have emitted at least once
          if (hasEmitted.every(Boolean)) {
            observer.next([...latestValues]);
          }
        },
      });
      subs.push(sub);
    });

    return () => subs.forEach(s => s.unsubscribe());
  });
}
```

Live Trace

name$ emits 'Naveen' at t=0, 'N.Kumar' at t=300. age$ emits 25 at t=150.

Time	Event
t=0	name$ emits 'Naveen'. latestValues = ['Naveen', undefined]. hasEmitted = [true, false] → not all true → no output
t=150	age$ emits 25. latestValues = ['Naveen', 25]. hasEmitted = [true, true] → all true → OUTPUT: ['Naveen', 25]
t=300	name$ emits 'N.Kumar'. latestValues = ['N.Kumar', 25] (age's slot untouched, keeps last value) → all still true → OUTPUT: ['N.Kumar', 25]

Key mechanism: it holds one "latest value" slot per source, and re-emits the entire combined array every time any single source fires — but only after every source has fired at least once.

Angular use case: a filter form with a dropdown and a search box — combine both into live query params, re-run search whenever either one changes, but wait until both have an initial value.

TOPIC 22 — forkJoin

RxJS Operator - Combination | Advanced | High

Waits for ALL sources to complete, emits only their FINAL values, once.

```
function customForkJoin(sources: Observable<any>[]): Observable<any[]> {
  return new Observable((observer) => {
    const finalValues: any[] = new Array(sources.length);
    const hasCompleted: boolean[] = new Array(sources.length).fill(false);
    const subs: Subscription[] = [];

    sources.forEach((source, index) => {
      const sub = source.subscribe({
        next: (value) => { finalValues[index] = value; },   // just overwrite - only
last value matters
        complete: () => {
          hasCompleted[index] = true;
          if (hasCompleted.every(Boolean)) {
            observer.next(finalValues);
            observer.complete();
          }
        },
      });
      subs.push(sub);
    });

    return () => subs.forEach(s => s.unsubscribe());
  });
}
```

Live Trace

userReq$ (HTTP call) completes at t=200 with {id: 1}. ordersReq$ (HTTP call) completes at t=500 with [order1, order2].

Time	Event
t=0	Both subscribed simultaneously. Both HTTP requests fire in parallel. No output yet
t=200	userReq$ emits {id: 1} → stored in finalValues[0]. Then complete() fires → hasCompleted = [true, false] → not all done → no output
t=500	ordersReq$ emits [order1, order2] → stored in finalValues[1]. Then complete() fires → hasCompleted = [true, true] → all done → OUTPUT: [{id:1}, [order1, order2]], then complete()

Key mechanism: ignores every intermediate next() except the last one before complete(), and only emits once, only after every single source has finished. If even one source never completes (e.g. a WebSocket that streams forever), forkJoin never emits at all — it's built for finite streams like HTTP calls, not ongoing ones.

Angular use case: loading a dashboard that needs user, orders, and settings all fetched before rendering anything — three parallel HTTP calls, render once all three are back.

combineLatest vs forkJoin

	combineLatest	forkJoin
Emits	Every time any source emits (after all have emitted once)	Once, only after all sources complete
Use case	Ongoing/live values that need to stay in sync	One-shot values (usually HTTP) that need to all finish
Works with infinite streams	Yes	No - never emits if any source never completes

TOPIC 23 — map

RxJS Operator - Transformation | Beginner | Critical

---

map doesn't create a new kind of Observable — it wraps the source Observable in a new one, whose producer subscribes to the original and re-emits transformed values.

```
function customMap<T, R>(source: Observable<T>, fn: (val: T) => R): Observable<R> {
  return new Observable<R>((observer) => {
    const sourceSub = source.subscribe({
      next: (value) => observer.next(fn(value)),   // transform, then forward
      error: (err) => observer.error(err),
      complete: () => observer.complete(),
    });

    return () => sourceSub.unsubscribe();   // teardown just cancels the source subsc
ription
  });
}
```

Notice: map's own Observable has no independent producer logic of its own — its entire job is subscribing to source and relaying, with one transformation step inserted. This is the pattern every pipeable operator follows — they're all just wrapper Observables around the original.

Live Trace

```
const nums$ = new Observable<number>((observer) => {
  observer.next(1);
  observer.next(2);
  observer.complete();
});

const doubled$ = customMap(nums$, x => x * 2);

const sub = doubled$.subscribe({ next: v => console.log(v) });
```

1. customMap(nums$, fn) called → returns a new Observable instance, doubled$. Its producer is the function shown above. Nothing runs yet - nums$'s producer hasn't fired either.

2. doubled$.subscribe({...}) called → doubled$'s producer runs → inside it, source.subscribe({...}) is called → this is nums$.subscribe(...) → now nums$'s own producer fires.

3. nums$'s producer calls observer.next(1) - but this observer is the inner one, the {next: value => observer.next(fn(value))...} object built inside customMap. So this call triggers fn(1) → 1 * 2 = 2 → then calls the outer observer.next(2) - which is your original {next: v => console.log(v)} - prints 2.

4. Same for observer.next(2) from nums$ → fn(2) = 4 → outer next(4) → prints 4.

5. nums$'s producer calls observer.complete() → inner complete handler calls outer observer.complete() → chain propagates upward.

6. customMap's producer returns () => sourceSub.unsubscribe(). If sub.unsubscribe() were called at this point, it would tear down doubled$'s subscription, which internally tears down nums$'s subscription too - cancellation propagates downward through the chain automatically.

Key insight: every operator in a .pipe(op1, op2, op3) chain is a nested Observable wrapping the one before it — subscribing flows outward-to-source, values flow source-to-outward, and unsubscribing flows outward-to-source again. map is the simplest possible case of this wrapping pattern — no timing logic, no branching, just transform-and-forward.

TOPIC 24 — filter

RxJS Operator - Transformation | Beginner | High

Same wrapping pattern as map — new Observable, producer subscribes to source, only difference is a conditional check before forwarding.

```
function customFilter<T>(source: Observable<T>, predicate: (val: T) => boolean): Obse
rvable<T> {
  return new Observable<T>((observer) => {
    const sourceSub = source.subscribe({
      next: (value) => {
        if (predicate(value)) {
          observer.next(value);   // only forward if predicate passes
        }
        // if predicate fails - nothing happens, value silently dropped
      },
      error: (err) => observer.error(err),
      complete: () => observer.complete(),
    });

    return () => sourceSub.unsubscribe();
  });
}
```

Live Trace

```
const nums$ = new Observable<number>((observer) => {
  observer.next(1);
  observer.next(2);
  observer.next(3);
  observer.next(4);
  observer.complete();
});

const evens$ = customFilter(nums$, x => x % 2 === 0);
evens$.subscribe({ next: v => console.log(v) });
```

1. customFilter(nums$, predicate) → returns new Observable evens$, producer stored, nothing runs.

2. evens$.subscribe({...}) → evens$'s producer runs → calls nums$.subscribe({...}) → nums$'s producer fires.

3. nums$ calls observer.next(1) (inner observer) → predicate(1) → 1 % 2 === 0 → false → observer.next() is never called - value dropped completely, outer subscriber sees nothing.

4. nums$ calls observer.next(2) → predicate(2) → true → inner calls observer.next(2) → outer next fires → prints 2.

5. nums$ calls observer.next(3) → predicate fails → dropped.

6. nums$ calls observer.next(4) → predicate passes → prints 4.

7. nums$ calls observer.complete() → propagates through unchanged → outer complete fires.

Key mechanism: filter never transforms the value, it only decides whether to call observer.next() at all. The "drop" isn't a special action — it's simply not calling the outer observer's next. No emission ever reaches downstream subscribers for a value that fails the predicate.

TOPIC 25 — tap

RxJS Operator - Transformation | Beginner | Medium

Same wrapper shape again — but tap never touches observer.next()'s argument. It runs a side-effect function, then forwards the original value untouched.

```
function customTap<T>(source: Observable<T>, sideEffectFn: (val: T) => void): Observa
ble<T> {
  return new Observable<T>((observer) => {
    const sourceSub = source.subscribe({
      next: (value) => {
        sideEffectFn(value);     // run side effect first - return value ignored
        observer.next(value);    // forward the SAME value, unmodified
      },
      error: (err) => observer.error(err),
      complete: () => observer.complete(),
    });

    return () => sourceSub.unsubscribe();
  });
}
```

Note the critical difference from map: map did observer.next(fn(value)) — used the return value. tap does sideEffectFn(value); observer.next(value) — completely ignores whatever sideEffectFn returns. That's the entire mechanical distinction between the two.

Live Trace

```
const nums$ = new Observable<number>((observer) => {
  observer.next(5);
  observer.complete();
});

const logged$ = customTap(nums$, x => console.log('side effect saw:', x));
logged$.subscribe({ next: v => console.log('subscriber got:', v) });
```

1. customTap(...) → new Observable logged$, producer stored, nothing runs.

2. logged$.subscribe({...}) → producer runs → nums$.subscribe({...}) → nums$'s producer fires.

3. nums$ calls observer.next(5) (inner observer) → First: sideEffectFn(5) runs → console.log('side effect saw:', 5) fires. Then: observer.next(5) - the exact same 5, not derived from the side effect's return - forwarded to outer observer → prints subscriber got: 5

4. nums$ calls complete() → propagates unchanged.

Why this matters practically: because tap's side-effect function's return value is discarded, whatever you do inside it — even return 999 — has zero effect on the stream. If you tried to mutate the stream from inside tap, you'd be fighting the mechanism; that's exactly why tap is reserved for logging/debugging/analytics, never for actual data transformation. That job belongs to map.

TOPIC 26 — scan

RxJS Operator - Transformation | Intermediate | High

Like Array.reduce, but it emits the running accumulator on every emission, not just once at the end. The producer holds state across calls — this is the key new mechanism here, since map/filter/tap had no memory between emissions.

```
function customScan<T, R>(source: Observable<T>, accumulatorFn: (acc: R, val: T) => R
, seed: R): Observable<R> {
  return new Observable<R>((observer) => {
    let accumulator = seed;   // state lives HERE, captured in closure, persists acro
ss emissions

    const sourceSub = source.subscribe({
      next: (value) => {
        accumulator = accumulatorFn(accumulator, value);   // update running total
        observer.next(accumulator);                         // emit the CURRENT runni
ng total, every time
      },
      error: (err) => observer.error(err),
      complete: () => observer.complete(),
    });

    return () => sourceSub.unsubscribe();
  });
}
```

The let accumulator = seed line is the whole mechanism. It's a variable in the producer's closure — created once, when subscribe() runs the producer, and every subsequent next() call from the source mutates that same captured variable.

Live Trace

```
const clicks$ = new Observable<number>((observer) => {
  observer.next(1);   // click value 1
  observer.next(1);   // click value 1
  observer.next(1);   // click value 1
  observer.complete();
});

const runningTotal$ = customScan(clicks$, (acc, val) => acc + val, 0);
runningTotal$.subscribe({ next: v => console.log('total:', v) });
```

1. customScan(...) → new Observable, producer stored. Nothing runs.

2. .subscribe(...) → producer runs → let accumulator = 0 executes right now, once. This value lives in memory for the lifetime of this subscription.

3. clicks$.subscribe(...) → source producer fires.

4. clicks$ calls observer.next(1) → accumulatorFn(0, 1) → accumulator reassigned to 1 → observer.next(1) fires outward → prints total: 1.

5. clicks$ calls observer.next(1) again → accumulatorFn(1, 1) → accumulator reassigned to 2 → outer next(2) → prints total: 2.

6. clicks$ calls observer.next(1) again → accumulatorFn(2, 1) → accumulator reassigned to 3 → prints total: 3.

7. complete() propagates.

Critical detail - per-subscription isolation. If a second, independent .subscribe() call happened on runningTotal$, scan's producer would run again from scratch, meaning a fresh let accumulator = seed — a brand new closure variable, starting back at 0. Two subscribers never share the same accumulator. This connects directly to the "each subscribe = independent producer run" rule — scan's state is just local variables inside that per-subscription execution, nothing global.

TOPIC 27 — debounceTime(ms)

RxJS Operator - Timing Control | Advanced | Critical

Introduces real timer-based control flow. The producer must track a pending timer, cancel it on every new emission, and only forward a value once the timer actually completes uninterrupted.

```
function customDebounceTime<T>(source: Observable<T>, ms: number): Observable<T> {
  return new Observable<T>((observer) => {
    let timerId: ReturnType<typeof setTimeout> | null = null;

    const sourceSub = source.subscribe({
      next: (value) => {
        if (timerId !== null) {
          clearTimeout(timerId);   // cancel the PREVIOUS pending emission
        }
        timerId = setTimeout(() => {
          observer.next(value);    // only fires if nothing cancelled it within ms
          timerId = null;
        }, ms);
      },
      error: (err) => observer.error(err),
      complete: () => {
        if (timerId !== null) clearTimeout(timerId);
        observer.complete();
      },
    });

    return () => {
      if (timerId !== null) clearTimeout(timerId);
      sourceSub.unsubscribe();
    };
  });
}
```

The mechanism: every new next() from the source cancels whatever timer is currently waiting, then starts a fresh one. A value only ever reaches the outer observer.next() if ms milliseconds pass with zero new emissions in between.

Live Trace — Search Box, debounceTime(300)

Source emits: 'a' at t=0, 'ab' at t=100, 'abc' at t=150, then nothing after.

1. t=0 - next('a') fires. timerId is null → no cancel needed. setTimeout(..., 300) started - scheduled to fire at t=300 with value 'a'.

2. t=100 - next('ab') fires. timerId is NOT null (still pending from step 1) → clearTimeout() runs → the t=300 firing for 'a' is killed, 'a' will never reach the subscriber. New setTimeout(..., 300) started - scheduled for t=400 with value 'ab'.

3. t=150 - next('abc') fires. Previous timer (due t=400) cancelled → 'ab' also killed, never emitted. New timer started, scheduled for t=450 with value 'abc'.

4. t=450 - no new emission happened between t=150 and t=450 (450ms window is actually 300ms from t=150) → timer fires uninterrupted → observer.next('abc') → subscriber finally receives 'abc'.

Only 'abc' ever reaches the subscriber. 'a' and 'ab' were scheduled but each got cancelled before their timer could complete, because a newer value arrived first.

Why this pairs with switchMap: debounceTime reduces "user is still typing" noise down to one settled value; switchMap then takes that single settled value and manages the HTTP call + cancellation for it. Doing it the other way (switchMap first) would fire and cancel HTTP requests on every keystroke instead of just the final one.

TOPIC 28 — throttleTime(ms)

RxJS Operator - Timing Control | Advanced | High

Opposite instinct from debounceTime. Instead of waiting for silence, it forwards the first value immediately, then enters a "cooldown" window where everything is dropped, until the window expires.

```
function customThrottleTime<T>(source: Observable<T>, ms: number): Observable<T> {
  return new Observable<T>((observer) => {
    let inCooldown = false;

    const sourceSub = source.subscribe({
      next: (value) => {
        if (!inCooldown) {
          observer.next(value);   // forward immediately - first value in the window
          inCooldown = true;
          setTimeout(() => {
            inCooldown = false;   // window expires - next value is now allowed throu
gh
          }, ms);
        }
        // if inCooldown is true - value is silently dropped, nothing scheduled for i
t
      },
      error: (err) => observer.error(err),
      complete: () => observer.complete(),
    });

    return () => sourceSub.unsubscribe();
  });
}
```

Key mechanical difference from debounceTime: there's no clearTimeout here. Nothing gets cancelled or rescheduled. inCooldown is a simple boolean gate — while true, every incoming value is just ignored outright, not queued, not delayed, gone.

Live Trace — Button Spam-Click Guard, throttleTime(300)

Source emits clicks at: t=0, t=50, t=100, t=350, t=380.

1. t=0 - next() fires. inCooldown is false → forward immediately → subscriber receives click #1. inCooldown set to true. setTimeout(..., 300) scheduled to flip it back at t=300.

2. t=50 - next() fires. inCooldown is true → dropped, nothing happens, no timer touched.

3. t=100 - next() fires. inCooldown still true → dropped.

4. t=300 - the scheduled timeout fires → inCooldown set back to false. Window closed. (No emission happens here - this timer only flips the gate, unlike debounce where the timer itself was the emission trigger.)

5. t=350 - next() fires. inCooldown is false (window reopened at t=300) → forward immediately → subscriber receives click #2. inCooldown set true again, new 300ms cooldown scheduled for t=650.

6. t=380 - next() fires. inCooldown is true → dropped.

Subscriber receives exactly 2 clicks: t=0 and t=350 — the first click in each 300ms window, everything else in between discarded.

Contrast with debounce, mechanically: debounce's timer is the thing that causes an emission (fires → emit). Throttle's timer only removes a block (fires → gate reopens, but doesn't emit anything itself). That's why debounce feels like "wait for quiet," and throttle feels like "rate limit."

TOPIC 29 — distinctUntilChanged

RxJS Operator - Timing Control | Intermediate | High

Simplest state mechanism of the bunch — the producer just remembers the last value it forwarded and compares.

```
function customDistinctUntilChanged<T>(source: Observable<T>): Observable<T> {
  return new Observable<T>((observer) => {
    let hasEmittedOnce = false;
    let lastValue: T;

    const sourceSub = source.subscribe({
      next: (value) => {
        if (!hasEmittedOnce || value !== lastValue) {
          lastValue = value;
          hasEmittedOnce = true;
          observer.next(value);   // forward - first value ever, OR different from la
st
        }
        // else: same as last value - silently dropped, lastValue stays unchanged
      },
      error: (err) => observer.error(err),
      complete: () => observer.complete(),
    });

    return () => sourceSub.unsubscribe();
  });
}
```

lastValue is a closure variable, same pattern as scan's accumulator — created once per subscription, persists across emissions, reset fresh for every new subscriber.

Live Trace

```
const status$ = new Observable<string>((observer) => {
  observer.next('idle');
  observer.next('idle');
  observer.next('loading');
  observer.next('loading');
  observer.next('loading');
  observer.next('done');
  observer.complete();
});

const changes$ = customDistinctUntilChanged(status$);
changes$.subscribe({ next: v => console.log(v) });
```

1. subscribe() runs producer → hasEmittedOnce = false, lastValue undefined.

2. next('idle') → hasEmittedOnce is false → condition true regardless of value → forward → prints idle. lastValue = 'idle', hasEmittedOnce = true.

3. next('idle') → hasEmittedOnce true, 'idle' !== 'idle' → false → dropped.

4. next('loading') → 'loading' !== 'idle' → true → forward → prints loading. lastValue = 'loading'.

5. next('loading') → 'loading' !== 'loading' → false → dropped.

6. next('loading') → same → dropped.

7. next('done') → 'done' !== 'loading' → true → forward → prints done.

Output: idle, loading, done — three prints instead of six, only transitions pass through.

Important gotcha: the default comparison is !== — strict equality. For objects/arrays, two different object references with identical contents will always count as "changed," even if every property matches, because !== compares references, not deep equality. That's the same reference-vs-value distinction from pass-by-value-vs-reference — distinctUntilChanged inherits that behavior directly since it's just doing a !== check under the hood. If you need deep comparison, you pass a custom comparator function as an argument, which replaces the !== line with your own logic.

TOPIC 30 — catchError

RxJS Operator - Error Handling | Advanced | Critical

New territory — first operator that reacts to the error channel instead of next. The mechanism: intercept the source's error() call, and instead of propagating it downward, swap in a replacement Observable.

```
function customCatchError<T>(source: Observable<T>, handler: (err: any) => Observable
<T>): Observable<T> {
  return new Observable<T>((observer) => {
    let currentSub: Subscription;

    currentSub = source.subscribe({
      next: (value) => observer.next(value),
      error: (err) => {
        const fallback$ = handler(err);      // build the replacement Observable
        currentSub = fallback$.subscribe({   // subscribe to IT instead
          next: (value) => observer.next(value),
          error: (err2) => observer.error(err2),   // if fallback itself errors, THAT
 propagates
          complete: () => observer.complete(),
        });
      },
      complete: () => observer.complete(),
    });

    return () => currentSub.unsubscribe();
  });
}
```

Critical detail: when error fires, the original source subscription is effectively done — nothing more will ever come from it (Observables guarantee error/complete are terminal, no further next() after either). So catchError's producer doesn't try to keep the original alive; it discards it and subscribes fresh to whatever Observable handler(err) returns.

Live Trace

```
const risky$ = new Observable<string>((observer) => {
  observer.next('data chunk 1');
  observer.error(new Error('network failed'));
  // note: no complete() call ever happens after error - terminal
});

const safe$ = customCatchError(risky$, (err) => {
  console.log('caught:', err.message);
  return of('fallback data');   // recovery Observable
});

safe$.subscribe({
  next: v => console.log('got:', v),
  complete: () => console.log('stream ended cleanly'),
});
```

1. .subscribe() on safe$ → producer runs → risky$.subscribe({...}) → risky$'s producer fires.

2. risky$ calls observer.next('data chunk 1') (inner observer) → forwards straight through → outer next fires → prints got: data chunk 1.

3. risky$ calls observer.error(new Error('network failed')) → this hits the error handler in customCatchError's inner subscription.

4. handler(err) runs → prints caught: network failed → returns of('fallback data') - a brand new Observable.

5. customCatchError immediately subscribes to that fallback Observable, reassigning currentSub.

6. of('fallback data')'s producer fires synchronously → observer.next('fallback data') → outer next fires → prints got: fallback data.

7. of(...) then calls complete() (it always does, single emission then done) → propagates to outer complete → prints stream ended cleanly.

Final output: got: data chunk 1, caught: network failed, got: fallback data, stream ended cleanly.

Why this matters practically: without catchError, that error() call would propagate straight to your component's error callback and the subscription would be dead — no further values, ever, even if you wanted to keep listening. catchError is the only mechanism that lets a stream survive an error by substituting a new source, rather than the subscription just terminating.

TOPIC 31 — retry(n)

RxJS Operator - Error Handling | Advanced | Critical

Similar territory to catchError — reacts to the error channel — but instead of substituting a different Observable, it re-subscribes to the same source, up to n times, before finally giving up and propagating the error.

```
function customRetry<T>(source: Observable<T>, maxRetries: number): Observable<T> {
  return new Observable<T>((observer) => {
    let attemptCount = 0;
    let currentSub: Subscription;

    function subscribeToSource() {
      currentSub = source.subscribe({
        next: (value) => observer.next(value),
        error: (err) => {
          if (attemptCount < maxRetries) {
            attemptCount++;
            console.log(`retry attempt ${attemptCount}`);
            subscribeToSource();   // re-subscribe to the SAME source, fresh
          } else {
            observer.error(err);   // retries exhausted - propagate for real
          }
        },
        complete: () => observer.complete(),
      });
    }

    subscribeToSource();   // initial attempt

    return () => currentSub.unsubscribe();
  });
}
```

Key mechanical detail: source.subscribe() is called again, fresh, every retry. Every .subscribe() call runs the producer from scratch, independently. So if source is an HTTP call, retrying means firing a genuinely new network request each time, not replaying old data.

Live Trace — retry(2) on a Flaky HTTP Call

```
let callCount = 0;
const flaky$ = new Observable<string>((observer) => {
  callCount++;
  console.log(`HTTP call #${callCount} sent`);
  setTimeout(() => {
    if (callCount < 3) {
      observer.error(new Error('server 500'));
    } else {
      observer.next('success data');
      observer.complete();
    }
  }, 100);
});

const resilient$ = customRetry(flaky$, 2);
resilient$.subscribe({
```

```
  next: v => console.log('got:', v),
  error: e => console.log('final failure:', e.message),
});
```

1. .subscribe() on resilient$ → producer runs → attemptCount = 0 → subscribeToSource() called → flaky$.subscribe(...) → HTTP call #1 sent.

2. t=100 - callCount is 1, < 3 → observer.error(new Error('server 500')) fires.

3. Inner error handler: attemptCount (0) < maxRetries (2) → true → attemptCount becomes 1 → prints retry attempt 1 → subscribeToSource() called again → fresh subscription → flaky$.subscribe(...) runs its producer again → callCount becomes 2 → HTTP call #2 sent.

4. t=200 - callCount is 2, still < 3 → error() fires again.

5. Inner error handler: attemptCount (1) < maxRetries (2) → true → attemptCount becomes 2 → prints retry attempt 2 → subscribeToSource() again → callCount becomes 3 → HTTP call #3 sent.

6. t=300 - callCount is 3, condition < 3 fails → observer.next('success data') fires → prints got: success data → complete() propagates.

If the server had kept failing past attempt 3: at the next error, attemptCount (2) < maxRetries (2) → false → observer.error(err) runs instead → prints final failure: server 500 — the original error finally reaches the subscriber, retries exhausted.

Practical note: retry alone retries immediately, back-to-back, no delay — can hammer a struggling server. In real Angular code you'd typically see retry({ count: 2, delay: 1000 }) or retry combined with a delay operator — same core mechanism, just with a setTimeout gap inserted before each subscribeToSource() call.

TOPIC 32 — take(n)

RxJS Operator - Subscription Management | Intermediate | Critical

---

Simplest mechanism yet — it just counts emissions, and calls unsubscribe() on itself once the count is hit. This is the operator that proves unsubscribe() can be triggered from inside the pipeline, not just by the end consumer.

```
function customTake<T>(source: Observable<T>, n: number): Observable<T> {
  return new Observable<T>((observer) => {
    let count = 0;

    if (n <= 0) {
      observer.complete();
      return () => {};
    }

    const sourceSub = source.subscribe({
      next: (value) => {
        count++;
        observer.next(value);
        if (count >= n) {
          observer.complete();       // tell downstream we're done
          sourceSub.unsubscribe();   // THIS is the key line - stop the source oursel
ves
        }
      },
      error: (err) => observer.error(err),
      complete: () => observer.complete(),
    });

    return () => sourceSub.unsubscribe();
  });
}
```

The line sourceSub.unsubscribe() inside the next handler is the entire mechanism. Nobody external called unsubscribe() — take calls it on itself, on its own subscription to the source, the moment it's satisfied. This runs the source's real teardown (closing a socket, clearing an interval) automatically, without the consumer ever lifting a finger.

Live Trace — take(3) on an Infinite Interval

```
const ticks$ = new Observable<number>((observer) => {
  let count = 0;
  const id = setInterval(() => observer.next(count++), 100);
  return () => {
    clearInterval(id);
    console.log('interval cleared');
  };
});

const firstThree$ = customTake(ticks$, 3);
firstThree$.subscribe({
  next: v => console.log('got:', v),
  complete: () => console.log('done'),
});
```

1. .subscribe() → customTake's producer runs → count = 0 → ticks$.subscribe(...) → ticks$'s producer fires → setInterval starts, firing every 100ms forever, no built-in stop.

2. t=100 - ticks$ emits 0 (inner observer) → count becomes 1 → outer next(0) → prints got: 0. 1 >= 3? No.

3. t=200 - emits 1 → count becomes 2 → prints got: 1. 2 >= 3? No.

4. t=300 - emits 2 → count becomes 3 → prints got: 2. 3 >= 3? Yes → observer.complete() fires → prints done → then sourceSub.unsubscribe() runs → this calls ticks$'s real teardown → clearInterval(id) → prints interval cleared.

5. t=400 onward - nothing. The interval is actually dead. Without take, this would tick forever into a subscriber that's already logically "done."

Why order matters in the code: observer.complete() is called before sourceSub.unsubscribe(). This ensures the downstream subscriber is told "no more values coming" through the normal channel first, and the cleanup of the actual resource happens right after — same two-step shutdown you'd get from manual unsubscribe(), just automated by a counter instead of a person calling it.

TOPIC 33 — takeUntil(notifier$)

RxJS Operator - Subscription Management | Advanced | Critical

---

Same self-unsubscribe mechanism as take, but instead of counting emissions, it subscribes to a second, independent Observable — the moment that one emits anything, the main subscription is torn down. This is the exact mechanism takeUntilDestroyed is built on.

```
function customTakeUntil<T>(source: Observable<T>, notifier: Observable<any>): Observ
able<T> {
  return new Observable<T>((observer) => {
    const sourceSub = source.subscribe({
      next: (value) => observer.next(value),
      error: (err) => observer.error(err),
      complete: () => observer.complete(),
    });

    const notifierSub = notifier.subscribe({
      next: () => {
        observer.complete();       // notifier fired - tell downstream we're done
        sourceSub.unsubscribe();   // kill the main subscription
        notifierSub.unsubscribe(); // kill the notifier subscription too - job done,
no longer needed
      },
    });

    return () => {
      sourceSub.unsubscribe();
      notifierSub.unsubscribe();
    };
  });
}
```

Two independent subscriptions running in parallel from the start: one to source, one to notifier. Whichever fires first — a next() from the notifier — triggers teardown of the other.

Live Trace — Real Angular Pattern, Component Destruction

```
const destroySignal$ = new Observable<void>((observer) => {
  // simulate: this fires when the component is destroyed
  setTimeout(() => observer.next(), 500);
});

const ticks$ = new Observable<number>((observer) => {
  let count = 0;
  const id = setInterval(() => observer.next(count++), 100);
  return () => { clearInterval(id); console.log('interval cleared'); };
});

const safeStream$ = customTakeUntil(ticks$, destroySignal$);
safeStream$.subscribe({
  next: v => console.log('got:', v),
  complete: () => console.log('completed'),
});
```

1. .subscribe() → customTakeUntil's producer runs → both subscriptions start immediately: ticks$.subscribe(...) starts the interval, destroySignal$.subscribe(...) starts its own 500ms timer.

2. t=100, t=200, t=300, t=400 - ticks$ emits 0, 1, 2, 3 → each forwarded straight through → prints got: 0 through got: 3.

3. t=500 - destroySignal$'s timer fires → calls observer.next() (on the notifier's inner observer) → this runs the notifier's next handler in customTakeUntil → observer.complete() fires → prints completed → sourceSub.unsubscribe() runs → ticks$'s teardown fires → clearInterval(id) → prints interval cleared → notifierSub.unsubscribe() also runs, cleaning up the notifier itself.

4. t=600 onward - nothing. Interval genuinely dead.

Direct line to takeUntilDestroyed(this.destroyRef): in real Angular code, destroySignal$ is effectively DestroyRef's internal destroy notification — an Observable-like source that emits exactly once when ngOnDestroy fires. takeUntilDestroyed is just takeUntil with that notifier pre-wired for you, so you don't have to manually build the destroy-signal Observable and pipe it in yourself.

TOPIC 34 — shareReplay(n)

RxJS Operator - Multicasting | Advanced | Critical

Completely different shape from everything so far. Every operator up to now built a new producer that re-runs from scratch on every .subscribe() — that's been the rule since the very first lesson. shareReplay breaks that rule on purpose: it makes multiple subscribers share one single underlying execution.

```
function customShareReplay<T>(source: Observable<T>, bufferSize: number): Observable<
T> {
  let sourceSub: Subscription | null = null;   // shared across ALL subscribers
  const buffer: T[] = [];                       // cached values, shared
  const observers: Observer<T>[] = [];          // every subscriber's callbacks, tra
cked here
  let isComplete = false;

  return new Observable<T>((observer) => {
    // replay whatever's already cached, immediately, to THIS new subscriber
    buffer.forEach(value => observer.next(value));
    if (isComplete) {
      observer.complete();
      return () => {};
    }

    observers.push(observer);

    // only subscribe to the REAL source ONCE, on the very first subscriber
    if (!sourceSub) {
      sourceSub = source.subscribe({
        next: (value) => {
          buffer.push(value);
          if (buffer.length > bufferSize) buffer.shift();
          observers.forEach(obs => obs.next(value));   // broadcast to EVERY subscrib
er
        },
        complete: () => {
          isComplete = true;
          observers.forEach(obs => obs.complete());
        },
      });
    }

    return () => {
      const idx = observers.indexOf(observer);
      if (idx !== -1) observers.splice(idx, 1);
      // note: real RxJS also has refCount logic to eventually unsubscribe sourceSub
      // when observers.length hits 0 - simplified here for clarity
    };
  });
}
```

Two structural changes from every prior operator: sourceSub, buffer, and observers are declared OUTSIDE the returned Observable's producer — not inside it. That means they're created once, when customShareReplay(...) itself is called, and persist across every subsequent .subscribe() call. Every subscriber pushes into the same observers array and reads from the same buffer,

instead of getting a fresh closure each time.

Live Trace — One HTTP Call, Two Subscribers

```
let callCount = 0;
const httpCall$ = new Observable<string>((observer) => {
  callCount++;
  console.log(`HTTP request #${callCount} sent`);
  setTimeout(() => {
    observer.next('user data');
    observer.complete();
  }, 200);
});

const cached$ = customShareReplay(httpCall$, 1);

cached$.subscribe({ next: v => console.log('Subscriber A got:', v) });

setTimeout(() => {
  cached$.subscribe({ next: v => console.log('Subscriber B got:', v) });
}, 300);   // subscribes AFTER the HTTP call already completed
```

1. customShareReplay(httpCall$, 1) called → sourceSub = null, buffer = [], observers = [] created once, right here.

2. cached$.subscribe(...) for Subscriber A → producer runs → buffer is empty, nothing to replay → isComplete false → A's observer pushed into observers → sourceSub is null → first time, so httpCall$.subscribe(...) runs → HTTP request #1 sent.

3. t=200 - httpCall$ emits 'user data' → pushed into buffer → observers.forEach(...) runs → only A is in the array → prints Subscriber A got: user data. Then complete() → isComplete = true → A's complete called.

4. t=300 - Subscriber B subscribes → producer runs → buffer now has ['user data'] → immediately replays it → prints Subscriber B got: user data - no HTTP call, no waiting. isComplete is true → B's complete() called right away too. sourceSub check is skipped entirely since it's already set.

Only ONE 'HTTP request sent' log, ever — despite two independent .subscribe() calls. Compare this to every other operator, where two subscribes always meant two independent producer executions. shareReplay is the deliberate exception: the state that normally lives inside the producer (per-subscription) has been hoisted outside it, into a scope shared by everyone.

Angular use case: a UserService.getCurrentUser() called from three different components on the same page — without shareReplay, that's three identical HTTP requests. With it, one request, three components getting the same cached response instantly.